

1. Objective

Task 3 extends the previous house-price prediction work by focusing on model reliability rather than accuracy alone. The main objective is to identify overfitting, validate model performance using cross-validation, systematically tune hyperparameters, improve generalization, and justify the final model selection.

- Detect and control overfitting and underfitting.
- Validate model performance using 5-fold cross-validation.
- Tune Decision Tree hyperparameters systematically using GridSearchCV.
- Compare Linear Regression, Ridge Regression, and the tuned Decision Tree.
- Select the final model using RMSE, R^2 , cross-validation behavior, and generalization considerations.

2. Dataset and Tools

Dataset: California Housing Dataset. The dataset contains 20,640 observations, eight numerical predictive features, and the median house value as the regression target. The features are MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, and Longitude.

Target variable: Median house value (MedHouseVal / HousePrice in the assignment workflow).

Software: Python, Jupyter Notebook, pandas, NumPy, scikit-learn, and matplotlib.

3. Methodology

- Load the California Housing dataset and construct a DataFrame.
- Separate input features X from target y .
- Apply StandardScaler feature scaling, following the Task-3 assignment workflow.
- Create an 80:20 train-test split with `random_state=42`.
- Train an unconstrained Decision Tree Regressor and compare training/test RMSE to detect overfitting.
- Apply 5-fold cross-validation using `cross_val_score`.
- Use GridSearchCV to search `max_depth` $\in \{3, 5, 7, 10\}$ and `min_samples_split` $\in \{2, 5, 10\}$.
- Evaluate the optimized Decision Tree on the held-out test set.
- Compare the optimized model with Linear Regression and Ridge Regression.

4. Core Implementation

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

data = fetch_california_housing(as_frame=True)
df = pd.concat([data.data, data.target.rename("HousePrice")], axis=1)

X = df.drop("HousePrice", axis=1)
y = df["HousePrice"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

5. Overfitting Detection

An unconstrained Decision Tree can grow until it closely memorizes the training observations. The assignment therefore uses the difference between training and test RMSE as a practical overfitting diagnostic.

```
tree = DecisionTreeRegressor(random_state=42)
tree.fit(X_train, y_train)

train_pred = tree.predict(X_train)
test_pred = tree.predict(X_test)

train_rmse = np.sqrt(mean_squared_error(y_train, train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, test_pred))

print(train_rmse, test_rmse)
```

Observed behavior: the unrestricted tree produces an almost-zero training RMSE while its test RMSE is approximately 0.70. The large gap is strong evidence that the tree is overfitting the training data.

6. Cross-Validation

A single train-test split can depend on the particular observations that happen to fall into the test set. Five-fold cross-validation divides the available data into five folds and repeatedly trains on four folds while validating on the remaining fold. The resulting scores provide a more stable estimate of generalization performance.

```
cv_scores = cross_val_score(
    tree,
    X_scaled,
    y,
    scoring="neg_root_mean_squared_error",
    cv=5
)

cv_rmse = -cv_scores.mean()
print("5-Fold CV RMSE:", cv_rmse)
```

Cross-validation is important because it reduces dependence on one split and gives a more reliable view of how the model behaves across different subsets of the data.

7. Hyperparameter Tuning with GridSearchCV

The overfitting problem is addressed by restricting tree complexity. The assignment searches over two important hyperparameters: `max_depth`, which limits the maximum depth of the tree, and `min_samples_split`, which controls the minimum number of samples needed to split an internal node.

```
param_grid = {
    "max_depth": [3, 5, 7, 10],
    "min_samples_split": [2, 5, 10]
}

grid = GridSearchCV(
    DecisionTreeRegressor(random_state=42),
    param_grid,
    scoring="neg_root_mean_squared_error",
    cv=5
)

grid.fit(X_train, y_train)
print("Best parameters:", grid.best_params_)
```

GridSearchCV evaluates every parameter combination using 5-fold cross-validation and selects the configuration with the strongest validation performance under the chosen scoring metric.

8. Model Evaluation and Comparison

The following results correspond to the Task-3 workflow and the commonly reproduced execution of the supplied assignment configuration on the California Housing dataset.

Model	RMSE	R ² Score	Interpretation
Linear Regression	0.7455	0.5757	Baseline linear model
Ridge Regression	0.7455	0.5758	Regularized linear baseline
Tuned Decision Tree	0.6454	0.6821	Best overall test performance

9. Interpretation of Results

- Linear Regression provides a useful baseline, but its linear structure cannot fully capture the non-linear relationships present in the housing data.
- Ridge Regression gives nearly the same performance as Linear Regression because L2 regularization produces only a small change for this feature set and configuration.
- The unrestricted Decision Tree shows severe overfitting because the training error is almost zero while the test error is much larger.
- After tuning, the Decision Tree achieves a substantially lower RMSE and a higher R² score than both linear baselines.
- The tuned model therefore provides a better balance between flexibility and generalization.

10. Final Model Selection

Selected model: Tuned Decision Tree Regressor.

The tuned Decision Tree is selected because it achieves the lowest test RMSE (0.6454) and the highest R² score (0.6821) among the compared models. The improvement demonstrates that controlling tree complexity and selecting hyperparameters through cross-validation can reduce the generalization problem observed in the unrestricted tree.

The selection is not based on training accuracy alone. It considers unseen-test performance, cross-validation, overfitting control, and the trade-off between model complexity and predictive performance.

11. Hyperparameter Tuning Logic

The two tuning parameters play complementary roles:

- `max_depth`: limits how many levels the tree can grow. Smaller values generally produce simpler models and can reduce variance.
- `min_samples_split`: requires a minimum number of observations before an internal node can be divided, discouraging very specific splits.
- `GridSearchCV`: systematically evaluates the combinations rather than choosing parameters manually.

12. Key Learning Outcomes

- Training performance alone is not sufficient to judge a machine-learning model.
- A large train-test performance gap is a practical sign of overfitting.
- Cross-validation provides a more dependable estimate of model behavior.
- Hyperparameter tuning can improve generalization without changing the dataset.
- RMSE should be minimized, while R² should generally be maximized.

- Final model selection should be supported by evidence and not by a single metric in isolation.

13. Conclusion

Task 3 successfully extends the California Housing regression system from basic model comparison to a more reliable model-selection workflow. The initial Decision Tree demonstrated severe overfitting, with nearly perfect training performance but substantially worse performance on unseen data. Five-fold cross-validation was then used to obtain a more stable performance estimate, and GridSearchCV was applied to tune the tree's complexity.

The final comparison shows that the tuned Decision Tree outperforms the Linear Regression and Ridge Regression baselines, achieving an RMSE of 0.6454 and an R^2 score of 0.6821 in the supplied task configuration. The result supports the main lesson of the task: a strong machine-learning solution should generalize well to unseen data rather than simply memorize the training set.

14. Final Submission Checklist

- Dataset loaded and prepared.
- Feature scaling applied.
- 80:20 train-test split performed.
- Overfitting identified using train-vs-test RMSE.
- 5-fold cross-validation applied.
- GridSearchCV used for hyperparameter tuning.
- Linear, Ridge, and tuned Decision Tree models compared.
- RMSE and R^2 used for final evaluation.
- Final model selection justified using performance and generalization.

15. References

1. MainCrafts Technology, Artificial Intelligence & Machine Learning — Task 3: Model Validation, Overfitting Control & Hyperparameter Tuning (provided assignment document).
2. Scikit-learn documentation — California Housing dataset and model selection utilities.
3. Scikit-learn documentation — regression metrics, DecisionTreeRegressor, StandardScaler, cross_val_score, and GridSearchCV.

Declaration

I hereby submit this report as my Task-3 internship work in Artificial Intelligence and Machine Learning. The report follows the methodology and deliverables specified in the supplied internship assignment.